

Lecture 7: More About Arrays and Integer Arithmetic

Table of contents

1	Objectives:	2
1.1	Why Learn Shift, Rotate, and Extended Arithmetic?	2
2	Shifting and Rotation	3
2.1	Logical and Arithmetic Shifts	4
2.1.1	Logical Shifts using SHL and SHR instructions	4
2.1.2	Arithmetic Shifts using SAL and SAR instructions	5
2.1.3	Flags Affected	6
2.2	Rotations	7
2.2.1	Rotations using ROL and ROR instructions	7
2.2.2	Rotations through carry using RCL and RCR	7
2.2.3	Flags Affected	9
2.3	Multi-precision Shifts	9
2.3.1	Flags Affected	10
2.4	Examples	11
2.5	Shift and Rotate Applications	13
3	Extended Addition and Subtraction	14
3.1	ADC Instruction	14
3.2	SBB Instruction	15
3.3	Flags Affected	15

3.4	Examples	15
4	Unpacked and Packed Decimal Arithmetic	17
4.1	AAA Instruction	18
4.1.1	Flags Affected	18
4.2	DAA Instruction	19
4.2.1	Flags Affected	19
4.3	AAS Instruction	19
4.3.1	Flags Affected	19
4.4	DAS Instruction	20
4.4.1	Flags Affected	20
4.5	AAM Instruction	20
4.5.1	Flags Affected	20
4.6	AAD Instruction	21
4.6.1	Flags Affected	21
4.7	Example	21
5	Manipulating Arrays	23
5.1	Repeated String Operations	25
5.2	Examples	25

1 Objectives:

In this lecture, you will learn:

1. Shifting and rotation
2. Extended addition and subtraction
3. Binary-Coded Decimal operations.
4. How to manipulate arrays using string primitive instructions.

1.1 Why Learn Shift, Rotate, and Extended Arithmetic?

These operations are fundamental to:

- **Performance optimization:** Bit operations are among the fastest CPU operations
- **Cryptography:** Rotation and shifting are core to encryption algorithms
- **Graphics programming:** Bit manipulation for color values and pixel operations
- **Network programming:** Extracting protocol header fields efficiently
- **Large number arithmetic:** Handling numbers beyond standard integer sizes

2 Shifting and Rotation

- Bit shifting means to move bits right and left inside an operand.
- x86 architecture provides a rich set of instructions in this area.

Instruction	Description
SHL	Shift (logic) left
SHR	Shift (Logic) right
SAL	Shift arithmetic left
SAR	Shift arithmetic right
ROL	Rotate left
ROR	Rotate right
RCL	Rotate through carry left
RCR	Rotate through carry right
SHLD	Double-precision shift left
SHRD	Double-precision shift right

2.1 Logical and Arithmetic Shifts

Syntax

SHL	<destination>, <count>
SHR	<destination>, <count>
SAL	<destination>, <count>
SAR	<destination>, <count>

Rules

- The destination operand can be either register or memory location
- The count operand can be either:
 1. CL register
 2. immediate (of size 8-bit)
 3. 1

- Therefore, the valid forms are:

```
SHL    reg/mem, imm8
SHL    reg/mem, CL
SHL    reg/mem, 1
```

- The form SHL reg/mem, 1 has its own, optimized op-codes in x86 architecture.
- The <count> is masked to 5 bits (or 6 bits with a 64-bit operand). The count range is limited to 0 to 31 (or 63 with a 64-bit operand).

2.1.1 Logical Shifts using SHL and SHR instructions

- If you treat the destination operand as unsigned integer, then use logical shifts.

- SHL and SHR instructions shift the bits of the destination operand to the left (toward most significant bit locations) and right (toward less significant bit locations), respectively.
- For each shift count to the right, the least significant bit of the destination operand is shifted into the CF flag, and the most significant bit is cleared. See Figure 1 (a).
- For each shift count to the left, the most significant bit of the destination operand is shifted into the CF flag, and the least significant bit is cleared. See Figure 1 (b).

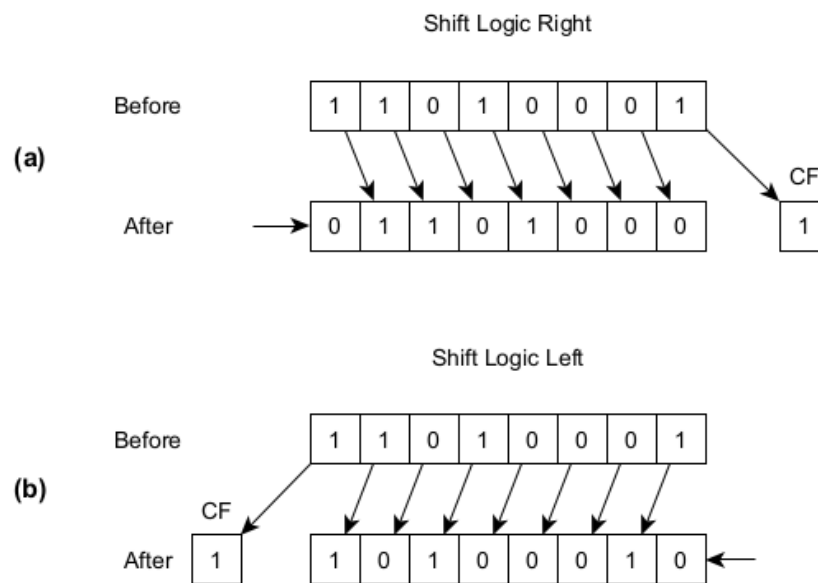


Figure 1

2.1.2 Arithmetic Shifts using SAL and SAR instructions

- If you treat the destination operand as signed integer, then use arithmetic shifts.
- For each shift count to the right, the least significant bit of the destination operand is shifted into the CF flag, and the most significant bit is set or cleared according to the sign of the original value. See Figure 2 (a).

- For each shift count to the left, the most significant bit of the destination operand is shifted into the CF flag, and the least significant bit is cleared. See Figure 2 (b).

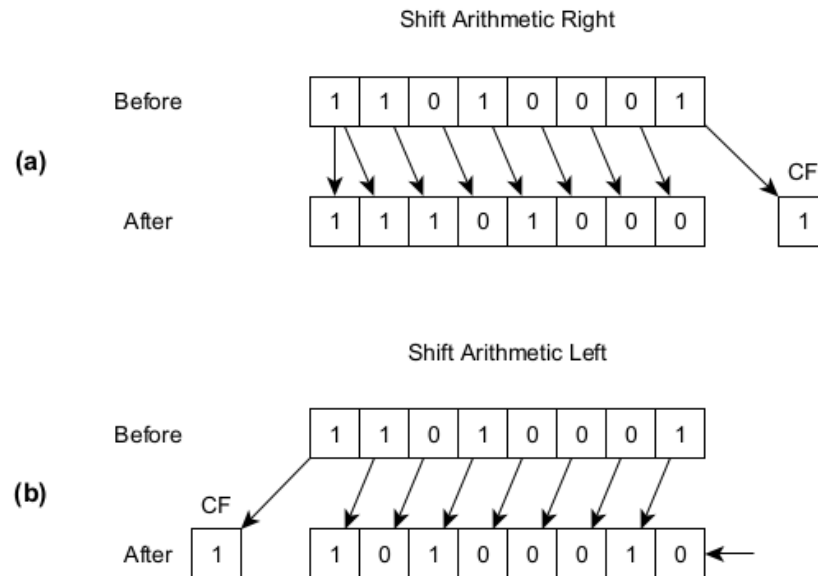


Figure 2

- Notice that there is no difference between SHL and SAL. Eventually, they have the same op-code.

2.1.3 Flags Affected

- The CF flag contains the value of the last bit shifted out of the destination operand.
- The OF flag is affected only on 1-bit shifts.
 - For left shifts, the OF flag is set to 0 if the most-significant bit of the result is the same as the CF flag (that is, the top two bits of the original operand were the same); otherwise, it is set to 1.
 - For the SAR instruction, the OF flag is cleared for all 1-bit shifts.
 - For the SHR instruction, the OF flag is set to the most-significant bit of the original operand.

- The SF, ZF, and PF flags are set according to the result.
- The AF flag is undefined.
- If the count is 0, the flags are not affected.

2.2 Rotations

Syntax

ROL	<destination>, <count>
ROR	<destination>, <count>
RCL	<destination>, <count>
RCR	<destination>, <count>

- The rotation instructions have the same rule as in shift instructions.
- The rotate left (ROL) and rotate through carry left (RCL) instructions shift ALL bits toward more-significant bit positions, except for the most-significant bit, which is rotated to the least-significant bit location.
- The rotate right (ROR) and rotate through carry right (RCR) instructions shift all the bits toward less significant bit positions, except for the least-significant bit, which is rotated to the most-significant bit location.

2.2.1 Rotations using ROL and ROR instructions

Figure 3 illustrates one-round of bits shifting. It shows that the bit shifted into CF is copied to the least or most significant bit (according to the shift direction).

2.2.2 Rotations through carry using RCL and RCR

Figure 4 illustrates the rotation through CF flag. If the direction of shifting is to the right, the CF is acting as least significant bit. If the direction of shifting is to the left, the CF is acting as most significant bit.

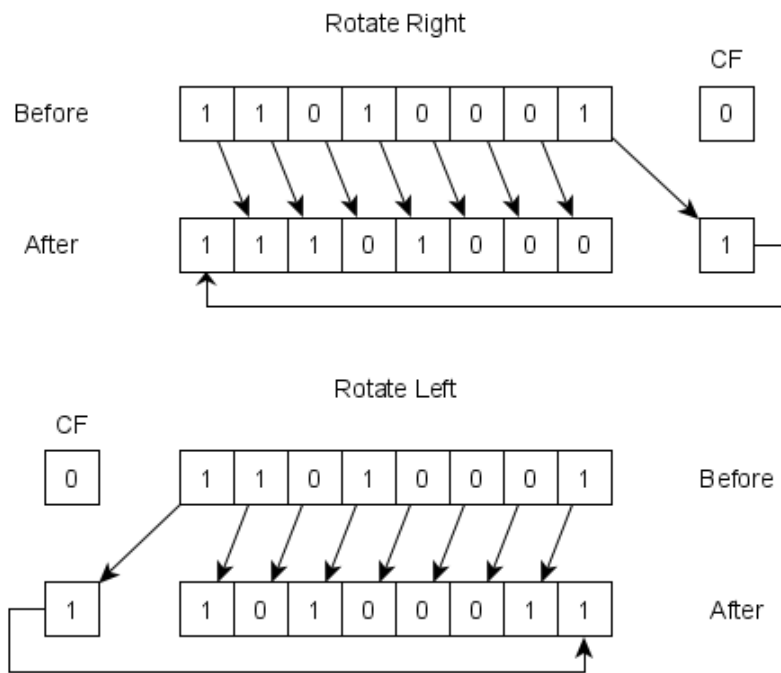


Figure 3

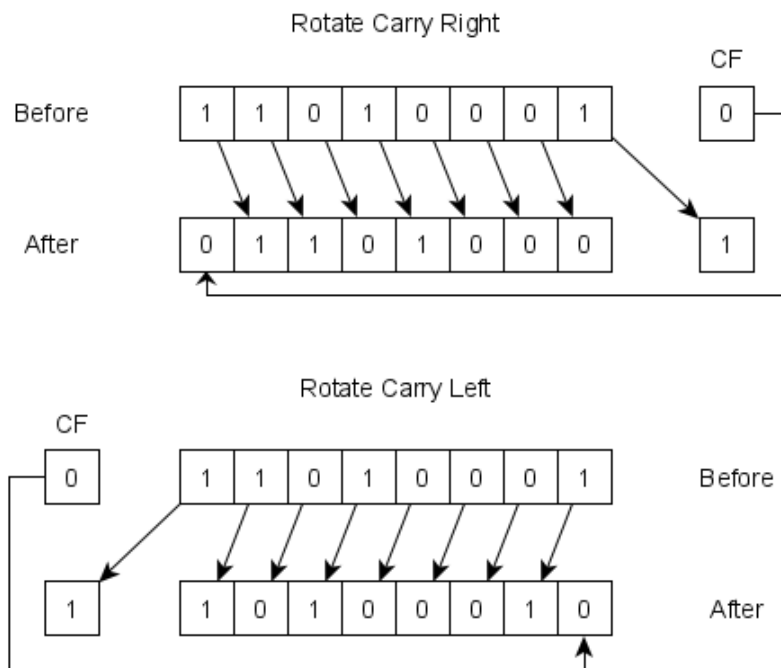


Figure 4

2.2.3 Flags Affected

- The CF flag contains the value of the last bit shifted out of the destination operand.
- The OF flag is defined only for 1-bit rotates:
 - For left rotates, the OF flag is set to the exclusive OR of the CF bit (after the rotate) and the most-significant bit of the result.
 - For right rotates, the OF is set to the exclusive OR of the two most-significant bits of the result.
- If count is 0, all flags are not affected.
- The SF, ZF, AF, PF flags are always undefined.

2.3 Multi-precision Shifts

Syntax

```
SHLD    <destination>, <source>, <count>
SHRD    <destination>, <source>, <count>
```

Rules

- The destination operand can be a register or a memory location of size word or doubleword.
 - The source operand must be a register
 - The destination and source operands must be of the same size.
 - The count operand can be unsigned immediate byte or CL register.
 - In 32-bit, the count value should be between 0 and 31.
- According to the rules above, the valid forms are:

```

SHLD    reg/mem16, 1
SHLD    reg/mem16, CL
SHLD    reg/mem16, imm8
SHLD    reg/mem32, 1
SHLD    reg/mem32, CL
SHLD    reg/mem32, imm8

```

- The SHLD instruction shifts the first operand (destination operand) to the left the number of bits specified by the count operand. The empty bit positions opened up by the shift are filled by the most significant bits of the source operand.
- Similarly, the SHRD instruction shifts the destination operand to the right, and the empty bit positions are filled by the least significant bits of the source operand.
- Figure 5 illustrates the movement of shifted bits from the source operand to the destination operand to the CF flag.



Figure 5

- Figure 6 shows the exact bits that will be shifted and moved from the source operand to destination operand when they are 32-bit.

2.3.1 Flags Affected

- The CF flag contains the value of the last bit shifted out of the destination operand.
- The OF flag is affected only on 1-bit shifts, the OF flag is set if a sign change occurred; otherwise, it is cleared.
- The SF, ZF, and PF flags are set according to the result.
- The AF flag is undefined.

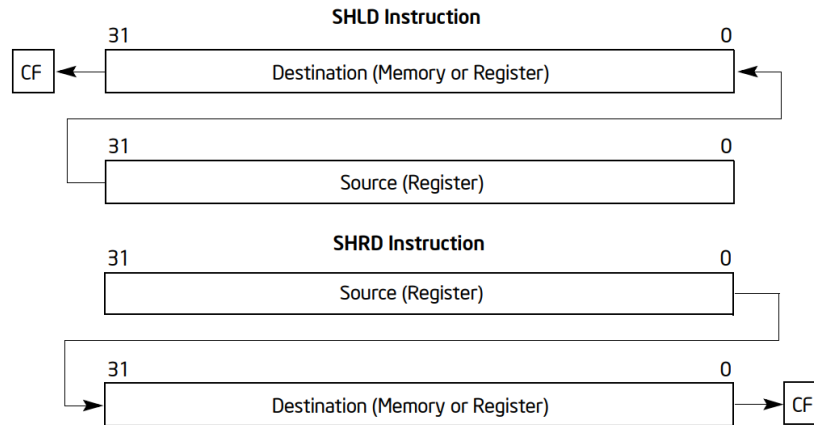


Figure 6

- If the count is 0, the flags are not affected. If the count is greater than the operand size, the flags are undefined.

2.4 Examples

Example 2.1.

```

mov    bl, 8Fh      ; BL= 1000 1111b
shl    bl, 1        ; BL= 0001 1110b, CF=1, OF=1

mov    bl, 8Fh
sal    bl, 2        ; BL= 0011 1100b, CF=0

```

Example 2.2.

```

mov    al, 0D7h     ; AL= 1101 0111b
shr    al, 2        ; AL= 0011 0101b, CF=1

mov    al, 0D7h
sar    al, 2        ; AL= 1111 0101b, CF=1

```

Example 2.3.

```

mov    al, 0C0h    ; AL= 1100 0000b
rol    al, 1       ; AL= 1000 0001b, CF=1
rol    al, 1       ; AL= 0000 0011b, CF=1
rol    al, 1       ; AL= 0000 0110b, CF=0

mov    al, 26h
rol    al, 4       ; AL= 62h

```

Example 2.4.

```

clc          ; Clear CF (CF=0)
mov    bl, 88h    ; BL= 1000 1000b
rcl    bl, 1      ; BL= 0001 0000b, CF = 1
rcl    bl, 1      ; BL= 0010 0001b, CF = 0

```

Example 2.5.

```

stc          ; Set CF (CF=1)
mov    ah, 10h    ; AL= 0001 0000b
rcr    ah, 1      ; AL= 1000 1000b, CF=0

```

Example 2.6.

```

mov    ax, 0
mov    bx, 1234h
shld   ax, bx, 4   ; AX=0001h, BX=1234h, CF=1

mov    ax, 0
shrd   ax, bx, 4   ; AX=4000h, BX=1234h, CF=0

```

2.5 Shift and Rotate Applications

1. The SHL and SAL instructions perform unsigned and signed multiplication when the multiplier is a power of 2. Shifting an integer n bits to the left multiplies it by 2^n .
2. The SHR and SAR instructions can be used to perform unsigned or signed division, respectively, of the destination operand by power of 2.
 - Using the SAR instruction to perform a division operation does not produce the same result as the IDIV instruction. The quotient from the IDIV instruction is rounded toward zero, whereas the “quotient” of the SAR instruction is rounded toward negative infinity.
3. The shift instructions can be used to extract data (field value) from a **bit string**.
 - For example, the first word of the Internet Protocol (IP) header has three fields: version, header length, and type of service, as shown below.

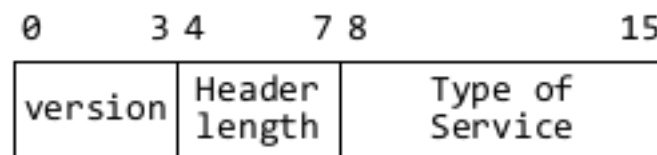


Figure 7

- Assume this word has been read and stored in AX register.
- Then, we can extract version by masking off bits from position 4 up to position 15:

```
mov    dx, ax           ; make a copy of AX
and    al, 00001111b    ; clear bits 4-7
mov    [version], al    ; store the result
```

- Then, we extract hdrlen (header length) by shifting original AX to the right 4 bits and then mask off bits from position 4 up to 15:

```
mov    ax, dx          ; restore original value
shr    ax, 4
and    al, 0000111b    ; clear bits 4-7
mov    [hdrlen], al
```

- Finally, we extract tos (type of service) by shifting original AX to the right 8 bits and then mask off bits from position 8 up to 15:

```
mov    ax, dx          ; restore original value
shr    ax, 8
mov    [tos], al
```

- Of course, we can simply assign DH directly to [tos] without shifting AX register. However, the purpose of the example is to show you how to extract data from a **bit string**.

3 Extended Addition and Subtraction

3.1 ADC Instruction

Syntax

```
ADC    <destination>, <source>
```

- ADC (Add with Carry) instruction adds the destination operand, the source operand, and the CF flag and stores the result in the destination operand.
- It performs the following operation: $DEST = DEST + SRC + CF$
- The rules are similar to ADD rules.

3.2 SBB Instruction

Syntax

```
SBB    <destination>, <source>
```

- SBB (Subtract with Borrow) instruction adds the source operand and the CF flag, and subtracts the result from the destination operand. The result of the subtraction is stored in the destination operand.
- Operation: $DEST = DEST - (SRC + CF)$
- The rules are similar to ADD rules.

3.3 Flags Affected

All flags are set according to the result.

3.4 Examples

Example 3.1. The following instructions add two 8-bit integers (FFh + FFh), producing a 16-bit sum in DL:AL, which is 01FEh.

```
mov     dl, 0
mov     al, 0FFh
add     al, 0FFh      ; AL=FFh + FFh = FEh, CF = 1
adc     dl, 0          ; DL=1, CF=0
```

Example 3.2. This example demonstrates a technique of adding two numbers having unlimited size. In C++, for example, no standard operator permits you to add two 1024-bit integers.

Let us assume, we have two numbers a and b that store their numbers as array of bytes of size n. The least significant byte is stored at index 0, and the most

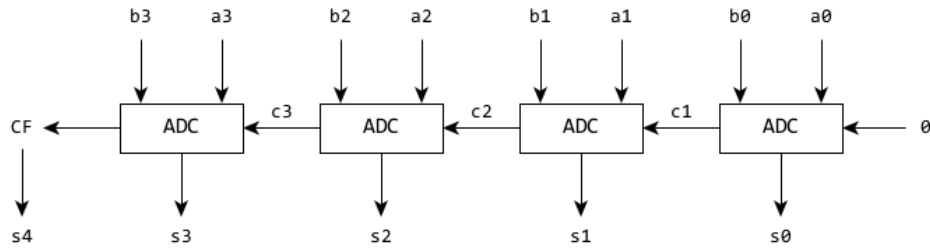


Figure 8

significant byte is stored at index $n-1$. The following diagram shows how to add two numbers (4-byte) and store the result in array s (5-byte).

Notice that the size of array s is $n+1$. The following code implements the **extended** addition:

```

mov     esi, a      ; ESI is pointing to array a
mov     edi, b      ; EDI is pointing to array b
mov     ebx, s      ; EBX is pointing to array s
mov     ecx, 128    ; a & b are 1024-bit, s is 1032-bit
clc                     ; set CF to 0
L1:
mov     al, [esi]    ; get the first integer a[i]
adc     al, [edi]    ; AL = a[i] + b[i] + CF
pushfd                ; save the carry flag
mov     [ebx], al    ; store partial result at s[i]
inc     esi          ; goto next integer in array a
inc     edi          ; goto next integer in array b
inc     ebx          ; goto next location in array s
popfd                ; restore the carry flag
loop    L1
; add any leftover carry
mov     BYTE [ebx], 0 ; set high byte to 0
adc     BYTE [ebx], 0 ; s[last] = s[last] + CF

```

4 Unpacked and Packed Decimal Arithmetic

- So far, we have studied arithmetic operations where the values are in binary representation. In this section, we study the IA-32's support for Binary-Coded Decimal (BCD).
- Recall that, in BCD, a decimal digit can be represented either as unpacked BCD or packed BCD. Please refer to [Lecture 1](#).
- x86 architecture provides a set of instructions for BCD integers. Eventually, the CPU always calculates in binary. It supports BCD arithmetic by adjusting the binary result into a BCD value.
- For example, if we need to add two BCD values 5 and 8, then,
- We first add them by using the binary addition instruction, such as ADD .

```
mov    ax, 5      ; AX = 0005h
add    ax, 8      ; AX = 000Dh
```

- Then, the CPU has an instruction to convert the result 000Dh to unpacked (i.e., AX = 0103) or packed (i.e., AX = 0013) BCD.
- Here are the list of instructions that support BCD conversions:

Mnemonic Description	
AAA	ASCII adjust after addition for unpacked BCD
AAS	ASCII adjust after subtraction for unpacked BCD
AAM	ASCII adjust after multiplication for unpacked BCD
AAD	ASCII adjust <u>before</u> division for unpacked BCD
DAA	Decimal adjust after addition for packed BCD
DAS	Decimal adjust after subtraction for packed BCD

- These instructions have no operand. The destination operand is implied either AL or AX register.
- In 64-bit mode, these instructions are not accessible.

4.1 AAA Instruction

- This instruction adjusts the sum of two unpacked BCD values to create an unpacked BCD result.
- The AL register is the implied source and destination operand.
- If the adjustment produces a decimal carry, the AH register is incremented by 1, and CF and AF flags are set.

Example 4.1. Consider the following code:

```
mov    ah, 0
mov    al, 4      ; AX = 0004h
add    al, 7      ; AX = 000Bh
aaa                    ; AX = 0101h
```

Example 4.2. We can also add two ASCII digits:

```
mov    ah, 0
mov    al, '8'    ; AX = 0038h
add    al, '2'    ; AX = 006Ah
aaa                    ; AX = 0100h, CF=1, AF=1
or     ax, 3030h  ; AX = 3130h = '10'
```

4.1.1 Flags Affected

- The AF and CF flags are set to 1 if the adjustment results in a decimal carry; otherwise they are set to 0.
- The OF, SF, ZF, and PF flags are undefined.

4.2 DAA Instruction

- This instruction adjusts the sum of two packed BCD values to create a packed BCD result.
- The AL register is the implied source and destination operand.

Example 4.3.

```
add    al, bl    ; before AL=79h, BL=35h
                ; after  AL=AEh, BL=35h CF=0
daa     ; AL=14h, CF=1, AF=1
```

4.2.1 Flags Affected

- The CF and AF flags are set if the adjustment of the value results in a decimal carry in either digit of the result.
- The SF, ZF, and PF flags are set according to the result.
- The OF flag is undefined.

4.3 AAS Instruction

- This instruction adjusts the result of the subtraction of two unpacked BCD values to create an unpacked BCD result.
- The AL is the implied source and destination operand.
- If the adjustment produces a decimal carry, the AH register is decremented by 1, and CF and AF flags are set.

4.3.1 Flags Affected

- Similar to AAA instruction

4.4 DAS Instruction

- This instruction adjusts the result of subtraction of two packed BCD values to create a packed BCD result.
- The AL register is the implied and destination operand.

4.4.1 Flags Affected

- Similar to DAA instruction.

4.5 AAM Instruction

- This instruction adjusts the result of multiplying two unpacked BCD values to create a pair of unpacked BCD values.
- The AX register is the implied source and destination operand.

Example 4.4.

```
mov    al, 8    ; AL=8h
mov    bl, 2    ; BL=2h
mul    bl       ; AX=10h
aam    ; AX=0106h
```

4.5.1 Flags Affected

- CF, OF, and AF flags are undefined.
- The SF, ZF, and PF flags are set according to the value in the AL.

4.6 AAD Instruction

- This instruction is different from the previous instructions. It adjusts two unpacked BCD digits stored in AX register to create a dividend value in binary format. Thus, when DIV is performed on the result, it will yield a correct unpacked BCD results.
- The AX register is the implied source and destination operand.

Example 4.5.

```
mov    ax, 0307h    ; dividend = decimal 37
aad                    ; AX = 0025h
mov     bl, 5        ; divisor
div     bl           ; AX = 0207h
                        ; quotient in AL
                        ; remainder in AH
```

- AAD instruction eventually sets the value in the AL register to $(AL + (10 \cdot AH))$, and then clears the AH register to 00H.

4.6.1 Flags Affected

- Similar to AAM.

4.7 Example

This example represents a complete assembly program that shows how to add two ASCII decimal values. The implementation is similar to Example 3.2, in which the carry from each digit addition is propagated to the next higher position.

```

1      global _main
2      extern _printf
3
4  DECIMALSZ EQU 10
5
6      section .data
7  d1      db "1234567890", 0
8  d2      db "7856341289", 0
9  prt_fmt db ` %10s\n+ %10s\n-----\n %11s\n`, 0
10     section .bss
11  d3      resb DECIMALSZ+2
12
13     section .code
14  _main:
15     mov     esi, DECIMALSZ-1    ; Set Index to Last Decimal
16     mov     ax, 0                ; clear AL and AH
17     mov     ecx, DECIMALSZ      ; set loop counter to DECIMALSZ
18  top1:
19     add     al, [d1 + esi]       ; AL = Carry + d1[i]
20     add     al, [d2 + esi]       ; AL = AL + d2[i]
21     aaa                     ; adjust to unpacked BCD
22     or      al, 30h             ; covert AL to ASCII
23     push    ax                  ; store result into stack
24     mov     al, ah              ; store Carry into AL
25     mov     ah, 0               ; clear AH
26     dec     esi                 ; move ESI to next highest position
27     loop    top1
28
29     ; the number of elements pushed onto stack is DECIMALSZ
30     mov     ecx, DECIMALSZ
31
32     ; Check if there is a carry out of the highest position
33     cmp     al, 0               ; if AL == 0h
34     je      done                ; jump if no carry
35     or      al, 30h             ; convert to ASCII
36     push    ax                  ; store onto stack

```

```

37         inc     ecx                ; add one element
38 done:
39         ; now move result from stack into d3
40         mov     esi, 0
41 top2:
42         pop     ax
43         mov     [esi + d3], al
44         inc     esi
45         loop    top2
46
47         mov     BYTE [esi + d3], 0 ; insert null-character
48         ; prints out the result
49         push    d3
50         push    d2
51         push    d1
52         push    prt_fmt
53         call    _printf
54         add     esp, 16
55
56         xor     eax, eax
57         ret

```

Here is the output:

```

1234567890
+ 7856341289
-----
9090909179

```

5 Manipulating Arrays

- The IA-32 has FIVE groups of instructions for processing arrays of bytes, words, and doublewords. These groups are known as **string primitives**.

Instruction Group	Description
MOVSx	<i>Move string</i> : copy data from memory addressed by ESI to memory addressed by EDI.
CMPSx	<i>Compare strings</i> : compare the contents of two memory locations addressed by ESI and EDI. It implicitly performs $[ESI] - [EDI]$
SCASx	<i>Scan string</i> : compare the accumulator register (AL, AX or EAX) to the contents of memory addressed by EDI. (i.e., $\text{Accumulator} - [EDI]$)
STOSx	<i>Store string</i> : store the accumulator register content into memory addressed by EDI.
LODSx	<i>Load string</i> : Load memory addressed by ESI into the accumulator register.

- The symbol x can be B for byte, W for word, D for doubleword, or Q for quadword (64-bit).
- Each string primitive instruction implicitly uses ESI, EDI, or both registers to address memory.
- References to the accumulator register imply the use of AL, AX, or EAX, depending on the instruction data size.
- After performing string primitive operation, the source index (ESI) and/or the destination index (EDI) are/is moved to the next memory address based on the CPU Direction flag (DF).
- If $DF = 0$, ESI or/and EDI are incremented. Otherwise, ESI or/and EDI are decremented.
- The instructions CLD and STD clear or set DF flag, respectively.

- String primitives can be repeated using a **Repeat Prefix** (explained next).

5.1 Repeated String Operations

- Each of string primitives performs one iteration of string operation.
- To operate on arrays, the string primitives can be combined with a repeat prefix (REP) to create a repeating instruction or be placed in a loop.
- The number of repetitions is determined by ECX register.
- The following repeated prefixes can be used in conjunction with a count in the ECX register to cause a string instruction to repeat:
 - REP: Repeat while ECX $\neq 0$.
 - REPE/REPZ: Repeat while ECX $\neq 0$ and ZF == 1.
 - REPNE/REPNZ: Repeat while ECX $\neq 0$ and ZF == 0.

5.2 Examples

Example 5.1 (Initialize an array). Given an array of short integers named arrX. We need to initialize all its elements to zero using string primitive:

```
cld          ; clear DF flag
mov  edi, arrX    ; destination
mov  ax, 0        ; initializer value
mov  ecx, 10      ; # of elements in arrX
rep  stosw        ; store string
```

Example 5.2 (Copy Arrays). In this example, let us translate the following for loop into assembly language using string primitives.

```

int x[100];
int y[100];
.
.
for (i=0; i<100; i++)
    y[i] = x[i];

```

The equivalent code in assembly language **without** using string primitive:

```

        mov     esi, x        ; the source array
        mov     edi, y        ; the destination array
        mov     ecx, 100      ; counter
top:     mov     eax, [esi]    ;
        mov     [edi], eax    ; y[i] = x[i]
        add     esi, 4        ; move to next elm in x
        add     edi, 4        ; move to next elm in y
        loop    top          ; repeat

```

Now, let us write the same code using string primitive:

```

        cld                ; clear direction flag
        mov     esi, x      ; ESI points to array x (source)
        mov     edi, y      ; EDI points to array y (target)
        mov     ecx, 100    ; repeat 100 times
        rep     movsd       ; copy string doublewords

```

Example 5.3 (Finding a target element). In this example, let us translate the following for loop into assembly language using string primitives.

```

int x[100];
int target = 8;
int found = 0;
.
.

```

```

for (i=0; i<100; i++)
    if (target == x[i]) {
        found = 1;
        break;
    }
if (found)
    printf("Target is found\n");
else
    printf("Target is not found\n");

```

The equivalent for-loop in assembly language **without** using string primitive:

```

        mov     esi, x          ; the source array
        mov     eax, 15         ; the target value
        mov     ecx, 100        ; counter
top:     cmp     eax, [esi]      ; compare
        je      FOUND           ; exit loop
        add     esi, 4          ; move to next elm in x
        loop    top             ; repeat
        ; not found -> print "Target is not found"
        .
        .
        jmp     DONE
FOUND:   ; target is found
        ; print "Target is found"
        .
        .
DONE:

```

Now, let us write the whole code using string primitive:

```

                section .data
arrX            dd      10, 13, 18, 14, 20, 11, 17, 18, 14, 12
ARRSIZE        EQU     ($-arrX)/4
fnd_msg         db      "Target is found", 13, 10, 0

```

```

not_fnd_msg db      "Target is not found", 13, 10, 0

        section .code
        .
        .

        cld                      ; clear DF flag
        mov     EDI, arrX        ; source array
        mov     EAX, 15          ; target
        mov     ecx, ARR_SIZE    ; counter
        repne   scasd
        je      FOUND
        ; not found
        push    not_fnd_msg
        jmp     PRINT
FOUND:   push    fnd_msg
PRINT:   call    _printf
        add     esp, 4
        .
        .

```

Example 5.4 (Count Vowels in a String). This example demonstrates LODSB to count the number of vowels in message string.

```

        section .data
message db      "Hello World", 0    ; must be null-terminated
vcounter dw     0                   ; vowel counter

        section .text
; main
        mov     esi, message
        xor     cx, cx

count_loop:
        lodsb                    ; load byte from [ESI] into AL
        test    al, al            ; check if we reach null terminator

```

```

        jz         done

        ; convert to uppercase by clearing bit 5
        ; Clear bit 5: 'a' (0110_0001) -> 'A' (0100_0001)
        and        al, 1101_1111b

        cmp        al, 'A'
        je         is_vowel
        cmp        al, 'E'
        je         is_vowel
        cmp        al, 'I'
        je         is_vowel
        cmp        al, 'O'
        je         is_vowel
        cmp        al, 'U'
        je         is_vowel

        jmp        count_loop      ; Not a vowel, continue
is_vowel:
        inc        cx              ; increment vowel counter
        jmp        count_loop
done:
        mov        vcounter, cx

```

Example 5.5 (Verify Password). This example demonstrates the use of the CMPSB (Compare String Byte) instruction to verify a user-input password against a stored password.

Initial Conditions:

- stored_password: null-terminated string containing the correct password.
- user_input: A null-terminated string containing the password attempt from the user.
- password_length: The known length of the correct password (excluding the null terminator).

Compare the two strings for an exact match up to password_length characters. If they match, set a result variable to 1; otherwise, set it to 0.

```
section .text

; main
cld
mov     esi, stored_password
mov     edi, user_input
mov     ecx, password_length

repe    cmpsb          ; repeat while equal
                        ; Stops when ECX=0 or bytes don't match

; check result
jz      password_match ; ZF=1 means all bytes are matched

; input does not match the password
mov     result, 0
jmp     end_verify
password_match:
mov     result, 1
end_verify:
        ; result now contains the verification result
```